

CleanNav: Deep learning and reinforcement strategies for smart robot exploration

K. Hemalatha^{*,} R. Tshewang, V. Tapaswi

School of Computer Science and Engineering, Vellore Institute of Technology, Chennai Campus, Chennai, Tamil Nadu 600127, India

ARTICLE INFO

Keywords:

YOLOv8
MobileNetV3
Object detection
Reinforcement learning
Robotic arm
Proximal Policy Optimization (PPO)

ABSTRACT

The developing trend in autonomous cleaning solutions for our homes has highlighted some challenges such as efficient navigation through spaces, avoiding obstacles, and interacting with objects in an intelligent way. Conventional vacuum cleaners adhere to set cleaning paths and thus do not adapt so well in changing scenarios. CleanNav integrates PPO-based reinforcement learning for adaptive navigation with a robotic arm for direct interaction in the environment, moving beyond vision-only or preplanned paths to enable end-to-end autonomy in dynamic homes. This paper proposes a novel hardware-based intelligent cleaning system, which combines a function of object interaction using robotic arm mounted on the vacuum cleaner base with reinforcement learning for navigation and deep learning for object detection. The system self-navigates and explores the surrounding by effectively avoiding obstacles with the help of ultrasonic sensor data fed to a model of reinforcement learning trained using Proximal Policy Optimization algorithm. The vision-based object detection module enhanced by the weighted voting approach, which fuses the models of YOLOv8 and MobileNetV3 ensure accurate real-time object detection. The hardware architecture of this model uses a robotic arm for object manipulation to allow uninterrupted cleaning. Complementing this, a lost & found feature assist users in tracing misplaced items based on logged detections by providing contextual hints about their last seen locations. With all these integration of robotic manipulation with intelligent exploration and navigation, cleaning efficiency is thus enhanced, making the system a dependable choice as a comprehensive home cleaning solution. Experimental results demonstrate the system achieves 96.4% mAP@50 in object detection using YOLOv8 and 99.2% classification accuracy with MobileNetV3. The robotic arm successfully picks up lightweight objects in 90% of attempts, while the weighted voting approach corrects critical misclassifications during real-time operation. Altogether, this implementation shows a very effective coordination between PPO-based navigation decisions and robotic arm actions, reflecting how reinforcement learning and mechanical control can operate together within a unified cleaning framework. Combining reinforcement learning for safe obstacle avoidance and exploration, this integration of vision, navigation and manipulation demonstrates robust system performance.

1. Introduction

In recent years, autonomous robotics have developed into fully-fledged advanced systems finding its way in the applications of household automation, such as robotic vacuum cleaners. These smart cleaning systems have gradually transitioned from pre-programmed movements to sophisticated AI-assisted systems capable of dynamically navigating the environment. However, one of the major remaining challenges is accuracy and efficiency of object detection in real-world environments. Conventional vacuum cleaners merely use generalized programs for obstacle avoidance, which do not have the capacity for object identification and classification [1]. Such limitations foster

under-performance in cleaning, possibly damaging items that are out of place in the house.

Deep learning has moved well beyond being a purely experimental tool and is now widely used in real-world, high-stakes applications. For example, vision models such as YOLO have shown reliable real-time performance in medical imaging tasks that support clinical diagnosis, where both detection accuracy and low latency matter [2]. Likewise, efficient architectures such as MobileNet are commonly used in precision agriculture for crop and disease monitoring, where models must run under limited compute and power while remaining robust in changing field conditions. The consistent success of these architectures

* Corresponding author.

E-mail addresses: hemalatha.kasirajan@vit.ac.in (K. Hemalatha), tshewang.rigzin2021@vitstudent.ac.in (R. Tshewang), vemuru.tapaswi2021@vitstudent.ac.in (V. Tapaswi).

<https://doi.org/10.1016/j.array.2026.100722>

Received 1 September 2025; Received in revised form 5 January 2026; Accepted 19 February 2026

Available online 27 February 2026

2590-0056/© 2026 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

across different domains highlights their practical reliability and efficiency, which motivates their use in consumer service robots that must perceive and react to dynamic household environments.

The proposed system therefore addresses these limitations through a hardware and software integrated system, merging reinforcement learning-based navigation of the robotic cleaner with object detection and robotic manipulation capabilities. At the core of its hardware is an Arduino Uno microcontroller that processes real-time data from ultrasonic sensors and an active IR sensor for adaptive navigation. An innovative approach to achieve the best object interaction functionality has been developed through the integration of a robotic arm controlled by servo motors, thereby proving this hardware framework reliable in executing high-level decisions derived from computational models. This work distinguishes itself through the integration of reinforcement learning-based adaptive navigation with a robotic arm capable of real-time object manipulation. Unlike conventional cleaning robots that isolate perception and movement, the proposed CleanNav system co-ordinates decision-making, motion control, and manipulation within a single PPO-driven framework. This integration highlights CleanNav as an end-to-end intelligent cleaning agent rather than an isolated detection system, demonstrating the effective coordination between learning-based navigation and physical interaction.

Enhancing object detection is crucial as it helps increase the learned behaviors of robots in cleaning so that they can recognize, differentiate, and manipulate different objects around them. Although the state-of-the-art deep learning model YOLO (You Only Look Once) brings excellent achievement for real-time object detection [3], one cannot fully rely on just a single detection paradigm for achieving the best results, especially in the context of small and less distinguishable objects. This warrants a need for a hybrid approach in which various models can be synergistically combined to maximize detection accuracy.

This research proposes a combined detection method where YOLOv8 model is integrated with MobileNetV3 model to improve the accuracy and efficiency of object detection in the autonomous vacuum cleaner system. The YOLOv8 structure is used for rapid object localization to yield bounding boxes with confidence scores, and then MobileNetV3, as a lightweight classification model demonstrated effective in resource-constrained environments [4], is utilized to further enhance the predictions for best accuracy. This hybrid approach leverages the advantages of both effective localization of YOLOv8 [5] and the accurate classification of MobileNetV3, thereby establishing a case for enhanced detection performance.

The navigation of the system is accomplished by the decisions of a reinforcement learning (RL) model, trained using Proximal Policy Optimization (PPO) algorithm chosen for its stability, sample efficiency, and applicability in continuous control tasks. The ultrasonic sensor data is fed as inputs to the trained model for making navigation decisions, in order to determine the next movement sequence of the robotic cleaner. By deploying on embedded hardware, the model controls the wheel motors to navigate around obstacles and optimize coverage for better exploration around the surrounding. Supplementing to that, the servomotor controlled robotic arm is programmed to execute pick-up and drop-off sequences for certain objects obstructing the path, thereby providing seamless integration between perception and action.

Balanced and careful coordination between its three essential modules — adaptive navigation, hybrid object detection, and robotic manipulation — aids in the successful realization of this integrated system. As the reinforcement learning model processes the sensor data to determine decisions of movement sequences, the object detection system classifies objects in the environment and only activates the robotic arm if certain predefined items are detected. This division of overall functionality allows for computational efficiency on the embedded hardware. The lightweight MobileNetV3 classifier, which has shown remarkable success in similar classification tasks [6], selectively activates after the initial detection from YOLOv8, while the pre-programmed

movement operations of the robotic arm reduces complex computational load. This hierarchical architecture is the backbone of the system's robustness, where the ultrasonic sensors ensure immediate obstacle avoidance, the RL model optimizes navigation and exploration, and the robotic arm supported by object detection module handle interaction tasks with detected items, thus ensuring reliable operation even under changing scenarios of home environment.

Complementing these capabilities, the lost & found system tracks and provides information regarding any detected objects. This benefits the users in home scenarios, where tiny items habitually disappear and they need to locate them. The system records objects detected on real-time and logs relevant details, including timestamps, object labels, confidence scores, and local contextual information regarding the nearby items. This data enables users to figure out the last-known location of the object for retrieval. The lost & found system adds to the overall functionality by employing object detection model and providing useful insights for daily usage.

The key contributions of the proposed work are as follows:

1. The implementation of an intelligent robotic arm to carry out selective pickup of predefined objects and relocate them at its left or right side depending on the availability of space, so that the cleaner can continue moving ahead to clean the area.
2. The integration of PPO-based reinforcement learning model trained for the robotic cleaner to self-navigate properly while avoiding obstacles and more importantly, to explore the surrounding effectively for maximum coverage of cleaning area.
3. A new hybrid object detection system that fuses YOLOv8 for fast localization and MobileNetV3 for accurate fine-grained classification in order to optimize object recognition for real-world household scenarios.
4. Application of the lost & found system developed through an effective logging mechanism to log detected objects in real-time which will give contextual information about the misplaced items when the user needs to locate them.

The next section of this paper comprises of an extensive survey of prior object detection algorithms, robotic vacuum cleaner systems, and hybrid detection techniques. Subsequently in Section 3, we will introduce our approach which describes the implementation of YOLOv8 and MobileNetV3 for object detection and classification, in addition to the reinforcement learning-based navigation system. We then evaluate the system's performance in Section 4, in terms of detection accuracy, object classification, and efficiency of the robotic arm in object handling. The findings demonstrate the efficacy of this system in real-time navigation, obstacle avoidance as well as selective object relocation with observations towards possible improvements. Ultimately, we aim to enhance the performance of autonomous cleaning systems using intelligent detection, object interaction, and adaptive navigation methods.

2. Related works

The established markets for cleaning robots have increased leaps and bounds, and researchers have actually invested time in advancing robot's navigation capability, energy efficiency, versatility, and user interactivity. Woohyeon Moon et al. [7] investigated RL use for optimization in a cleaning path for autonomous robots. Their research applied the Proximal Policy Optimization (PPO) algorithm in combination with Transfer Learning (TL) and Reward Shaping (RS) techniques for enhancing the learning proficiency in various environments. The proposed system performed more effectively in path planning, consumed less power, and showed better flexibility than conventional systems. Similarly, Arjun M et al. [8] implemented Simultaneous Localization and Mapping (SLAM) with Robot Operating System (ROS) thereby realizing real-time path planning and navigation in addition to obstacle avoidance. Their research highlighted the importance of efficient

mapping and path planning for autonomous cleaning robots through its cost-effectiveness and efficiency for cleaning per performance.

Deep learning-based perception has proven useful well beyond domestic robotics, especially in areas where decisions must be both accurate and timely. In medical imaging, Pacal et al. [9] introduced a real-time polyp detection method built on YOLO, illustrating that YOLO-style detectors can combine strong detection performance with the low latency needed for diagnostic support. In precision agriculture, Alswilem and Asadov [10] compared several CNN architectures for maize disease classification and showed that lightweight models such as MobileNetV2 can retain high recognition accuracy while keeping computational demands low, an important requirement for deployment on resource-limited platforms. Together, these results suggest that YOLO and MobileNet are not arbitrary choices, but established and reliable architectures that transfer well across domains, making them appropriate for real-time perception in consumer service robots operating in cluttered household environments.

In their survey, Tan et al. [11] found that coverage outcomes differ across classical and heuristic Coverage Path Planning (CPP) approaches for indoor robots. In addition to this, Zhao and Hwang [12] combined SLAM with Bidirectional A* in ROS and reached near-complete coverage in a structured 8 × 8 m setup, underscoring the strengths of planned, map-based navigation.

In the field of object detection and navigation, Jonghoek Kim [13] introduced a new vacuum cleaning system consisting of two linked robots: a Cleaner Robot (CR) and a Storage Robot (SR). The CR was specifically implemented to clean up tight areas, while the SR provided improved suction power and the capacity to hold dust. Obstacle detection and path-planning strategies were shown to play significant roles in planning the motion of tethered robots. Manasa M et al. [14] designed an affordable autonomous vacuum cleaner employing ultrasonic sensors to detect obstacles and Arduino UNO for management. Their system exhibited efficient navigation and dusting, thus demonstrating quite a lot of cost-effectiveness in automated cleaning.

In robotic manipulation, Kumra et al. [15] report strong performance with a deep-learning grasping model (GR-ConvNet v2), with 98.8% grasp accuracy on the Cornell dataset and over 95% success in real-world tests on a 7-DOF Baxter arm, reflecting the advantages of calibrated hardware and stable laboratory conditions. By contrast, Xiao et al. [16] evaluate grasping in outdoor agricultural settings and observe 82%–87% success, where lighting changes, occlusions, and irregular object geometry make the task more difficult. Taken together, these studies illustrate the trade-off between controlled laboratory performance and robustness under real-world settings, which helps frame the design space for low-cost mobile domestic manipulators such as CleanNav.

Navigation that involves interacting with objects, often discussed under navigational rearrangement or Navigation Among Movable Obstacles (NAMO), has recently gained attention because it requires navigation and manipulation to work together rather than as separate modules. Yang et al. [17] present a hierarchical reinforcement learning approach in which a mobile manipulator can actively push or move obstacles when a clear path is not available, allowing the robot to progress through cluttered scenes by combining high-level decisions with low-level control. In a related household setting, Iyer et al. [18] study everyday object rearrangement, where a mobile manipulator navigates across rooms, detects misplaced items, and picks and relocates them as part of long-horizon home tasks. Gu et al. [19] further frame rearrangement as a core embodied-AI problem and propose a multi-skill framework that decomposes long tasks into reusable navigation and manipulation skills in cluttered indoor environments. Although these works typically rely on high-end mobile manipulators and benchmark-oriented setups, they collectively reinforce the key idea that object interaction can be essential for successful navigation. In contrast, CleanNav explores this idea from a low-cost consumer robotics perspective, using selective object relocation to maintain smooth navigation

and cleaning continuity, rather than aiming for full semantic scene rearrangement or task-level reorganization.

Energy efficiency and noise reduction have also been major research areas. Ki Yong An et al. [20] proposed the use of acoustic metamaterials (AMM) to decouple vacuum cleaner noise from suction performance. The research used Helmholtz resonators to create a metastructure that was found to successfully suppress noise within given frequency bands. Vikas Singh Panwar et al. [21] developed a solar-powered floor-cleaning robot that was capable of both wet and dry cleaning. The robot's energy-saving design and autonomous navigation proved it effective for multiple cleaning purposes.

Human–robot interaction and user experience have also been explored in cleaning robots. Nur Beril Yapici et al. [22] have investigated householders' interaction with robot vacuum cleaners with the focus on customization and user-centered design. Their findings concluded that users lean more towards having customizable options as opposed to purchasing new models, which points toward the need for personalization of robotic vacuum technology. Sophie Alice Grimme et al. [23] developed a cat-shaped vacuum cleaner for use in care homes, especially among dementia patients. Residents were found to engage the robot like a pet, showing the promise of zoomorphic designs in improving emotional health.

Recent research has focused on cost-effective and multiple-purpose cleaning robots. Anil Eren and Hatice Doğan [24] created a low-cost vacuum cleaning robot with navigation features. The system used ultrasonic and infrared sensors for the avoidance of obstacles and had multi-pattern cleaning capability, providing a low-cost substitute for commercial units. Raj Vishal et al. [25] designed a two-function cleaning robot to clean both the floor and the walls. The robot's capacity to switch smoothly between functions exhibited its efficiency and versatility.

Also, Zhenjing Li et al. [26] discussed window-cleaning robots in an integrated way, dividing them according to application environments and examining their key technical parts. The paper brought out difficulties of developing useful and secure robots for high-altitude cleaning and reported different mobility methods and adhesion mechanisms. Prachi Ambadkar et al. [27] suggested an AI-based smart cleaner based on IoT with obstacle detection and hand gesture control for enhanced user experience. The dual-mode operation of the system (auto and manual) highlighted the advantages of using open-source hardware and software to make automation affordable.

Additionally, Yuda Irawan et al. [28] designed an automatic floor cleaner robot employing ultrasonic sensors and Arduino Uno for navigation. The feature of the system to sense the obstacles and come up with instantaneous decisions proved the potential of ultrasonic-based navigation. Jason Lin et al. [29] investigated the operation of cordless handheld vacuum cleaners with focus on particulate emissions and their impact on indoor air quality. The study set the significance of standardized methodology in motor development and filter seal in a bid to reduce risks to health.

Lastly, Shabnam FakhrHosseini et al. [30] explored the development of smart home technologies and suggested a taxonomy of home automation based on increasing levels of intelligence and user interaction. The research identified the main obstacles to the adoption of smart homes as usability problems and privacy issues, and suggested a formal framework to enhance industry cooperation.

The collective advancements in robotic cleaning systems — spanning navigation algorithms [7,8], energy-efficient designs [20,21], and user-centric interfaces [22,23] — provide critical foundations for our integrated approach. Drawing particular inspiration from RL-based navigation [7] and hybrid detection systems, we introduce a novel approach of enhanced object recognition through weighted-voting mechanism integrated with PPO-optimized RL based navigation and robotic manipulation capabilities which are absent in prior implementations. Moving forward from the current approaches, our system integrates real-time object interaction and adaptive cleaning functionalities through the lost and found tracking system.

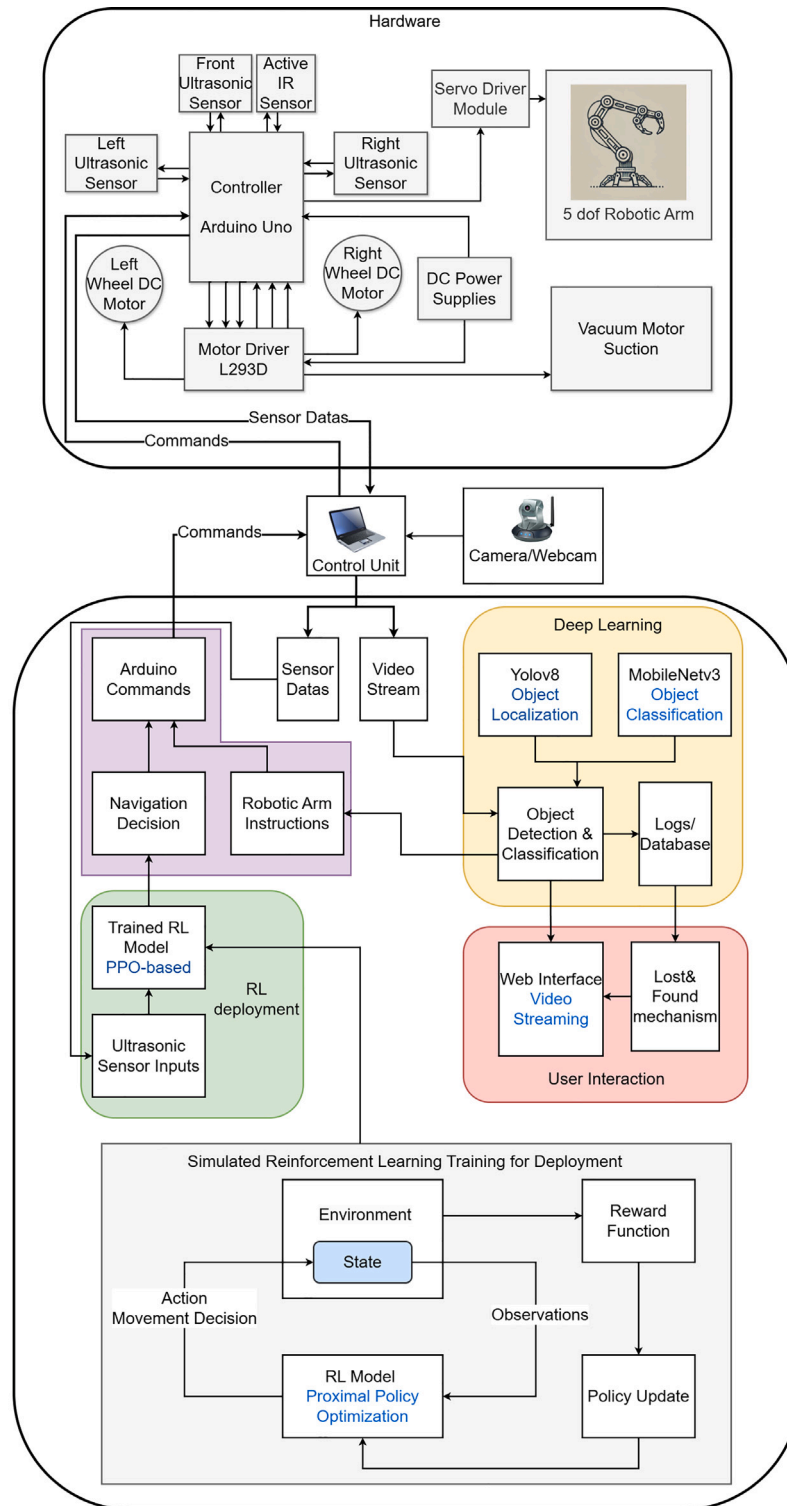


Fig. 1. System Architecture Diagram.

3. Methodology

3.1. System architecture overview

The proposed autonomous vacuum cleaning system is constructed using an integrated model that combines navigation, object detection, robotic arm control, and human interaction through a lost & found feature. The system aims at offering smooth operation by enabling

diverse components to engage in a coordinated way, ensuring efficient cleaning, intelligent object detection, and selective object handling.

3.1.1. System architecture and operational framework

The robotic cleaner is designed to perform multiple tasks simultaneously, by integrating different object detection models and an RL model with the hardware platform that includes a mounted robotic arm as presented in the architecture design in Fig. 1.

The fundamental functionality of this cleaner involves reinforcement learning-based navigation for autonomous traversal. Obstacles are detected and safe navigation is ensured using ultrasonic sensors. In addition to this, there is a camera installed on the vacuum cleaner to provide real-time vision and data capture to facilitate object detection and classification.

Initially, the system navigates in a home environment based on the PPO-based reinforcement learning (RL) model that has been trained in simulation for real-time navigation decisions. Mounted with ultrasonic sensors at the front, left, and right sides, the robotic system constantly measures distances to nearby obstacles, whereby these sensor readings are fed into the RL model. Based on this input, the RL model decides to go forward, turn left, or turn right according to how it learned to make decisions during its simulated training, aiming for maximum floor coverage while also avoiding collisions with objects or obstacles. To ensure proper safety, if an obstacle appears too close suddenly, the RL decision is overridden, and the system is programmed to turn towards the direction with more free space.

As the robot progresses with its navigation, its onboard camera captures live video, which is processed by a hybrid deep-learning pipeline for object detection. Firstly, YOLOv8 scans the scene at a high speed, drawing bounding boxes around the objects that comes into the view of the camera. The system crops each detected region and passes it to MobileNetV3, a lighter but more precise model, for fine-grained classification. The two models' predictions are combined via a weighted voting system to get the more accurate and reliable decision about the detected item.

When the system identifies an object from its predefined pickup list, it commands the robotic arm for object handling. The robot first halts its RL-based navigation and adjusts its position until the object is perfectly centered in the camera's view. The robotic arm then performs its motion sequences of pick-up and drop-off actions on the object to displace it aside and move forward to clean the newly accessible floor area which was otherwise blocked by that object.

All detected objects are logged with timestamps and labels, which plays important role in the lost & found functionality, where users can obtain information on lost objects based on their last recognized location in the household setting which is hinted through the knowledge of nearby objects detected at the same time when the lost item's detection was last logged. For better user experience, a web interface is built to display real-time video streaming and search through the log of recognized objects.

The system relies on Arduino Uno microcontroller to handle the hardware operations such as motor control, sensor data acquisition, and movement of the robotic arm. Simultaneously, a control system running in the background on python is responsible for all the computations and decision making for navigation choices, object detection, and system management.

3.1.2. Component integration

Integration of hardware and software is necessary for stability of the system as well as for high performance. The primary areas of the integration include:

- **Navigation System Integration:** Ultrasonic sensors are installed at the front, left, and right sides of the cleaner, and the active IR sensor is mounted at the bottom of the front side. The data from the sensors are sent in real time so as to support instantaneous decision-making during navigation by the trained RL model.
- **Object Detection and Classification Integration:** Live video footage from the camera is sent to the object detection module, which consists of the YOLOv8 and MobileNetV3 models, for detection and classification of objects. The detected objects' real-time data is logged, which supports the lost & found system.

- **Robotic Arm Integration:** The robotic arm will function based on the commands received to the Arduino from the object detection module whether to pick the item detected or ignore. The arm is programmed to pick specific objects such as socks, crumpled paper, spectacles or other small obstacles and place them at its left or right side depending on the availability of space, so that the cleaner can continue moving ahead to clean the area.
- **System Coordination and Communication:** Commands are transmitted fast and accurately, thereby eliminating any delay and error, through serial communication between the python-based control system and the Arduino Uno microcontroller.

With these integrations, various modules communicate in synchronization, performing their varied functions and thus, enhancing overall efficiency of the system in achieving desirable output and optimized performance.

3.2. Reinforcement learning-based navigation

Algorithm 1 Simulation Algorithm for Reinforcement Learning-Based Navigation

Input: Environment, robot, obstacles, episode length.
Output: Observation, reward, termination status, additional info.

```

1: Initialize: Load environment, robot, obstacles; initialize grid; set episode length and step counter.
2: for each timestep  $t$  in episode do
3:   Get sensor data:  $front\_distance, left\_distance, right\_distance$ .
4:   Obstacle Avoidance:
5:     if  $front\_distance < 0.2$  then Turn left if  $left\_distance > right\_distance$ , else turn right.
6:   else Continue current action.
7:   end if
8:   Execute Movement:
9:     if action == 0 then Set target velocities: [5.0, 5.0].    ▷ Forward
10:    else if action == 1 then Set target velocities: [-5.0, 5.0].    ▷ Turn left
11:    else if action == 2 then Set target velocities: [5.0, -5.0].    ▷ Turn right
12:    end if
13:    Apply motor control.
14:    Update Environment: Step simulation; get robot's new position  $(x, y)$ .
15:    Reward Calculation:
16:      if  $(x, y)$  is new then  $R \leftarrow R + 2.0$     ▷ Exploration reward
17:      else  $R \leftarrow R - 0.2$     ▷ Revisiting penalty
18:      end if
19:      if collision then  $R \leftarrow R - 5.0$ ; terminate episode.    ▷ Collision penalty
20:      end if
21:      if coverage  $\geq 70\%$  then  $R \leftarrow R + 5.0$     ▷ High coverage bonus
22:      end if
23:      Check Termination:
24:        if  $step\_count \geq episode\_length$  or collision then Terminate episode.
25:        end if
26:      Return: Observation, reward  $R$ , termination status, collision status.
27:    end for
28: Reset Environment: Clear grid; reset robot and obstacles; return initial observation.

```

The autonomous vacuum cleaner's navigation system is structured to function optimally in dynamic home environments by utilizing a reinforcement learning strategy as illustrated through Algorithm 1.

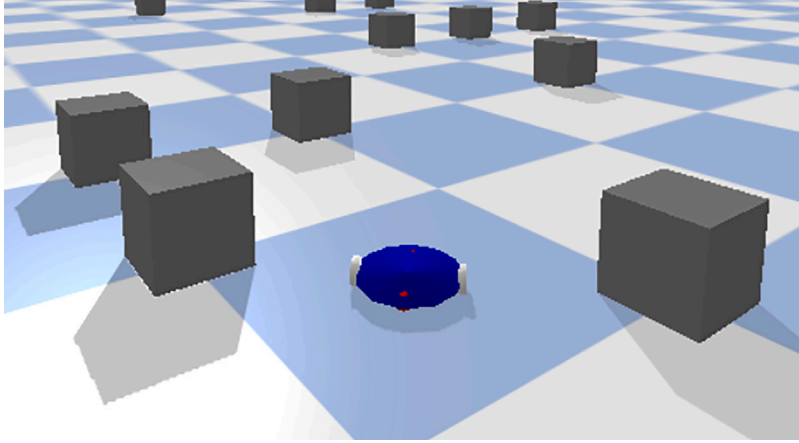


Fig. 2. Simulated robotic vacuum in the PyBullet environment.

Through its exploration behavior, the reinforcement learning policy enables the robot to encounter and detect objects during navigation, indirectly supporting later manipulation tasks triggered by the vision module. In contrast to traditional path-following techniques using predefined routes, this system applies a trained RL model, allowing the vacuum cleaner to make navigation decisions from sensor data in real-time. The adaptive strategy provides the vacuum cleaner with better capabilities for navigating through spaces efficiently without colliding onto objects and increasing coverage of areas to be cleaned.

3.2.1. Model selection: Proximal Policy Optimization (PPO) algorithm

The PPO algorithm was chosen as the core reinforcement learning model because it is stable, sample efficient, and applicable to continuous control tasks. PPO is a policy gradient approach that balances stable learning and good exploration by restraining extreme updates to the policy. PPO works by using a surrogate objective function that controls and limits the divergence between the new policy and the old policy. This is done with a clipping mechanism that stops the model from undergoing aggressive updates during training, so that the changes in policy is regulated carefully. The PPO objective function is given as:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right] \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ represents the probability ratio between the new and old policy, A_t is the advantage estimate, and ϵ is the clipping parameter. This formulation guarantees that changes are kept in a limited range, avoiding sudden spike in policy changes at the same time while ensuring progressive learning.

The PPO algorithm was used due to its capacity to provide efficient real-time decision-making while exploring different cleaning paths. The model was trained to drive the vacuum cleaner's movements by reading sensor inputs and taking suitable actions. The action space for the robot includes three main move commands: Move Forward — the vacuum cleaner moves forward while consistently checking obstacle distance; Turn Left — the vacuum cleaner turns left by a given angle; and Turn Right — the vacuum cleaner turns right by a specified angle, to prevent colliding with obstacles.

By integrating these movement commands with a suitable reward structure, the PPO model learns optimal navigation strategies efficiently. The reward system is structured to promote efficient room coverage, obstacle avoidance, and smooth movement, making the vacuum cleaner work well in real-world scenarios.

The reward function gives the total rewards obtained at the end of each episode, which could be both positive or negative depending on the majority of rewards or penalties the robot faced, and it can be computed as:

$$R_t = w_1 \cdot \text{exploration_reward} + w_2 \cdot \text{coverage_bonus} - w_3 \cdot \text{revisiting_penalty} - w_4 \cdot \text{collision_penalty} \quad (2)$$

Where:

- R_t is the total reward at time step t .
- The weights w_1, w_2, w_3, w_4 represent the robots' accumulated number of rewards or penalties throughout its episode.
- The exploration_reward encourages the robot to move towards unexplored areas.
- The coverage_bonus rewards the robot for covering more areas at a particular episode.
- The revisiting_penalty penalizes the robot for revisiting an area.
- The collision_penalty penalizes the robot for colliding with obstacles.

The robust generalization of the PPO algorithm in diverse settings and layouts of the room makes it suitable for this application, aimed at improving the flexibility of the robotic cleaner as well as its performance in highly repeated cleaning activities.

3.2.2. Simulation environment for RL training

PyBullet physics engine was employed to provide a realistic simulation environment for robot navigation strategy development and testing, as shown in Fig. 2. Differential drive kinematics were implemented in the design of the robot model, along with ultrasonic sensor-based obstacle detection to imitate real-world conditions. Obstacles were also added to simulate normal indoor clutter to create an environment resembling close to real-world scenarios, which maintained the robustness of the model. The fast and adaptive environment of PyBullet simulation allowed for efficient pre-deployment testing, priming the model for hardware integration.

3.2.3. Training process

The reward function was formulated to promote efficient exploration and avoidance of obstacles, thereby forming a good foundation for effective learning during the training. A positive reward of +2.0 was provided for going to new areas, and a minor penalty of -0.2 was implemented for visiting the same place repeatedly. Collision carried

a high penalty of -5.0 along with episode reset. Moreover, if the robot covered over 70% area, a reward of $+5.0$ bonus was provided. Training was executed for 500,000 timesteps, where hyperparameters were tuned to realize stable learning as well as convergence.

3.2.4. Deployment of the trained RL model on hardware

The trained RL model was deployed on the hardware system using an Arduino Uno for motor control and sensor input. The ultrasonic sensors of the robot gives real-time distance readings, which are used in navigation according to the learned policy. The movement commands for the robot were converted to motor control signals through an L293D motor driver IC. Meticulous mapping of the simulated environment to the real-world setup helped in effectively transferring and deploying the learned behavior of the trained robotic model to the real world settings.

3.2.5. Obstacle avoidance strategy

The robot's avoidance strategy used a combination of reactive and learned behaviors. During the training, the robot learned to avoid the obstacles while concurrently optimizing exploration. The robot is making decisions for left or right turns using sensor input with the following rule:

$$\text{turn_direction} = \begin{cases} \text{left} & \text{if } d_{\text{left}} > d_{\text{right}} \\ \text{right} & \text{otherwise} \end{cases}$$

Where:

- d_{left} is the distance measured by the left ultrasonic sensor.
- d_{right} is the distance measured by the right ultrasonic sensor.

In addition, a safety feature was introduced to override learned behavior if a critical barrier was sensed within 10 cm. In such scenario, the robot dynamically adjusted its path by turning in the direction where more space was available, instead of following the decision made by the trained RL model. And if the environment is safe from obstacles, the model was implicitly trained to explore more random areas which helps in coverage of the space adequately. This hybrid strategy of functioning through both the actions of following the commands led by the RL model's decision and overriding those operations with direct decisions based on sensor data at critical stage of obstacle avoidance, led to the improvement of the robot's performance in navigating dynamic and cluttered spaces with enhanced safety and efficiency.

The reinforcement learning policy will govern the robot's spatial decision-making, while the robotic arm functions as the physical actuator that carries out contextual interactions in the environment, so that it performs its cleaning tasks with less hindrance from the obstacles. This connection between navigation intelligence and manipulation defines CleanNav's primary innovation, allowing perception-driven actions that adapt to dynamic household conditions.

3.3. Object detection and classification

The object detection and classification mechanism is one of the main functions of the vacuum cleaner system proposed in this study since it supports object detection, classification, and interaction with different objects in the scene. The module combines YOLOv8 for object localization and MobileNetV3 for classification, both offering verification to each others' decision regarding detected item, to facilitate accurate object identification in complex and dynamic home environments.

3.3.1. Image acquisition and pre-processing for model dataset input

As dataset preparation is very important for the overall output of the model, a custom dataset was designed for the training and evaluation of the object detection algorithm targeting 16 relevant household objects for lost-item retrieval. The images were collected through:

- Web sources — A very diverse set of images was collected actively and systematically through specific searches, ensuring variations in object shape and design, views and occlusions. For example, the object might look closed or open for spectacles or keys partially blocked by hands or bags.
- Custom captures — These were taken in contrasting home environments, either under controlled conditions or natural lighting, using a 12MP RGB camera. The acquired images were intended to simulate real-world deployment scenarios.

Effective image pre-processing techniques were employed to improve model performance and data consistency.

- For YOLOv8, images were resized to 640×640 pixels to satisfy the specifications of the model architecture.
- The images were also resized to 224×224 pixels to comply with MobileNetV3's input requirements.
- Before the training was done, the dataset also underwent pixel intensity normalization and data augmentation methods like random rotation, flip, and change in brightness for model robustness.

These pre-processing techniques enhanced the system's efficiency in object detection and recognition from various lighting conditions, clutter, and different camera view angles.

3.3.2. YOLOv8 for object localization

YOLOv8 was selected for object detection because of its good trade-off between speed, accuracy, and efficiency. The single-stage architecture of the model allows for fast object detection and is best suited for real-time applications such as our cleaning robot.

A YOLOv8 model was specifically trained using a dataset of 16 various object categories such as household items like keys, remote controls, crumpled paper, and pencil shavings — objects relevant to the tasks of vacuuming and some objects based on cleaner's ability to pick up, in order to test the overall functionality of our system. The data were annotated via bounding boxes in order to localize the objects with precision in the working environment of the robot.

The YOLOv8 model was trained for a total of 100 epochs, with a batch size of 16. At inference time, YOLOv8 yielded bounding box coordinates, class labels, corresponding confidence levels for each identified object. This data allowed the system to discover object obstacles and take the corresponding actions, such as initiating robotic arm movement or activating obstacle avoidance features.

3.3.3. MobileNetV3 for fine-grained classification

For better accuracy in object recognition, particularly for small or visually similar objects, MobileNetV3 was integrated with the YOLOv8 model. MobileNetV3 was chosen due to its lightweight architecture to facilitate real-time deployment on low-power hardware.

A MobileNetV3 model was trained independently on the same customized dataset. This specific model was found to be extremely efficient in identifying slight differences between objects that may look visually identical but are actually not the same — e.g., distinguishing between 'pencil shavings' and 'dust,' and a 'remote control' and a 'cell phone'.

The integration of MobileNetV3 with YOLOv8 led to improved classification accuracy making the object detection module more reliable, allowing accurate detection of even those object classes that have resemblance to each other and might otherwise confuse the model.

3.3.4. Weighted voting for final prediction

Algorithm 2 YOLO + MobileNet Weighted Voting

Input: Image or video frame I .

Output: Final predicted class c^* and bounding box B for each detected object.

- 1: Load YOLOv8 model M_{yolo} .
 - 2: Load MobileNet model $M_{mobilenet}$.
 - 3: Run M_{yolo} on I :
 - 4: Get bounding boxes $B_i = (x_1, y_1, x_2, y_2)$.
 - 5: Get confidence scores s_i^{yolo} .
 - 6: Get class IDs c_i^{yolo} .
 - 7: For each detected object i :
 - 8: Crop region R_i from I using B_i .
 - 9: Preprocess R_i for $M_{mobilenet}$.
 - 10: Run $M_{mobilenet}$ on R_i :
 - 11: Get class probabilities $P_i^{mobilenet}(c)$.
 - 12: Get confidence score $s_i^{mobilenet}$.
 - 13: Get predicted class $c_i^{mobilenet} = \arg \max_c P_i^{mobilenet}(c)$.
 - 14: Calculate weights w_i^{yolo} and $w_i^{mobilenet}$:
 - 15: $w_i^{yolo} = \frac{s_i^{yolo}}{s_i^{yolo} + s_i^{mobilenet}}$
 - 16: $w_i^{mobilenet} = \frac{s_i^{mobilenet}}{s_i^{yolo} + s_i^{mobilenet}}$
 - 17: Combine predictions:
 - 18: Compute weighted confidence for each class c :
 - 19: $\text{weighted_confidence}_i(c) = w_i^{yolo} \cdot \text{confidence}_{yolo}(c) + w_i^{mobilenet} \cdot \text{confidence}_{mobilenet}(c)$
 - 20: Select the final predicted class:
 - 21: $c_i^* = \arg \max_c \text{weighted_confidence}_i(c)$
 - 22: Draw bounding box B_i and label c_i^* on I .
 - 23: Repeat for all detected objects in I .
 - 24: Display processed frame I in real-time.
 - 25: Stop if 'q' is pressed.
-

A weighted voting mechanism was implemented that combined the strengths of the two models. Since YOLOv8 excels in fast object localization while MobileNetV3 excels in complex classification, the system uses both model's confidence scores to generate the final prediction. The process is formalized by Algorithm 2 as shown above.

The weighted voting mechanism accommodate decisions of both the models while also giving attention to the more confident model. For each object detected, weights are computed from the confidence score of each model of YOLOv8 and MobileNetV3:

$$w_{yolo} = \frac{\text{confidence}_{yolo}}{\text{confidence}_{yolo} + \text{confidence}_{mobilenet}} \quad (3)$$

$$w_{mobilenet} = \frac{\text{confidence}_{mobilenet}}{\text{confidence}_{yolo} + \text{confidence}_{mobilenet}} \quad (4)$$

It uses a weighted combination of the confidence scores from the two model predictions to find the final predicted class, by selecting the highest weighted confidence to make the decision.

$$c^* = \arg \max_{c \in C} (w_{yolo} \cdot \text{confidence}_{yolo}(c) + w_{mobilenet} \cdot \text{confidence}_{mobilenet}(c)) \quad (5)$$

where,

- C : Set of all possible classes.
- $\text{confidence}_{yolo}(c)$: Confidence score for class c from the model trained using YOLOv8.
- $\text{confidence}_{mobilenet}(c)$: Confidence score for class c from the model trained using MobileNetV3.
- w_{yolo} and $w_{mobilenet}$: Computed weights from eqn. (3) and (4).

This approach can reduce false alarms to a large extent while improving object recognition in mission-critical applications like finding

lost objects or separating small pieces of unwanted items from valuable ones.

Fig. 3 presents the confidence voting mechanism that mediates between predictions from YOLOv8 and MobileNetV3 models. The flow begins with YOLOv8 conducting a preliminary pass of detections (bounding boxes and class scores) that are subsequently adjusted by MobileNetV3 for ambiguous cases through fine-grained classification. The decision contribution from the two models is dynamically re-scaled according to its normalized confidence score, with more certain predictions having the higher-weighted influence. The fusion occurs per detected object after cropping and resizing the region (Algorithm 2, Lines 7–16), where the final class is determined with maximized localization accuracy from YOLOv8 and classification accuracy from MobileNetV3.

3.4. Hardware and communication setup

The underlying hardware architecture of this robotic cleaner system as presented in Fig. 4 is designed to provide real-time object detection, navigation through the environment, and robotic arm manipulation. This system revolves around an Arduino Uno microcontroller by integrating the ultrasonic and IR sensors for obstacle avoidance, standard resolution RGB camera for object recognition, and servomotor controlled robotic arm for effective interaction with detected items. The communication between the Arduino and the python-based processing system is facilitated through serial communication via UART protocol, which allows sending Arduino commands to control actuation and sensor data as inputs to the trained models efficiently. Improved power distribution and motor control facilitate continuous and seamless performance in dynamic home environments.

3.4.1. Power and compute configuration

In the current prototype, vision processing and decision-making run on an external host computer, while the Arduino Uno is responsible for sensor interfacing and actuator control via serial communication. To avoid stressing the Arduino's onboard power regulation, the robotic arm servos are driven through a PCA9685 module with independent servo power, whereas the base motors are controlled using an L293D motor driver. The vision pipeline operates on a 480p (640 × 480) RGB stream to maintain real-time performance while retaining sufficient visual detail for common household objects. Together, these design choices improve system stability and clearly separate actuation-related power demand from the compute load of perception in the current setup.

3.4.2. Core hardware components

Microcontroller

Arduino Uno is the microcontroller used in this system for motor control, sensor data readings, and movement of the robotic arm, Arduino Uno is incredibly versatile with its I/O pins by allowing it to connect with easily with different components. Data from the sensors is fetched by this microcontroller and processed to issue commands to the motors and send inputs to python-based processing models vial serial communication.

Sensors

- **Ultrasonic Sensors (HC-SR04)**: Three sensors are mounted at the front, left, and right of the robot, and they help in giving clear information on obstacles in the movement path of the robot so that it can maneuver around them without any collision.
- **IR Sensor**: The infrared sensor is placed on the front bottom region of the robot to help detect staircases and edges so that it avoids falling down.
- **Camera**: The 480p RGB camera with a resolution of 640 × 480 pixels is the primary visual sensor for object detection and classification and it is placed on top of the robotic chassis to obtain

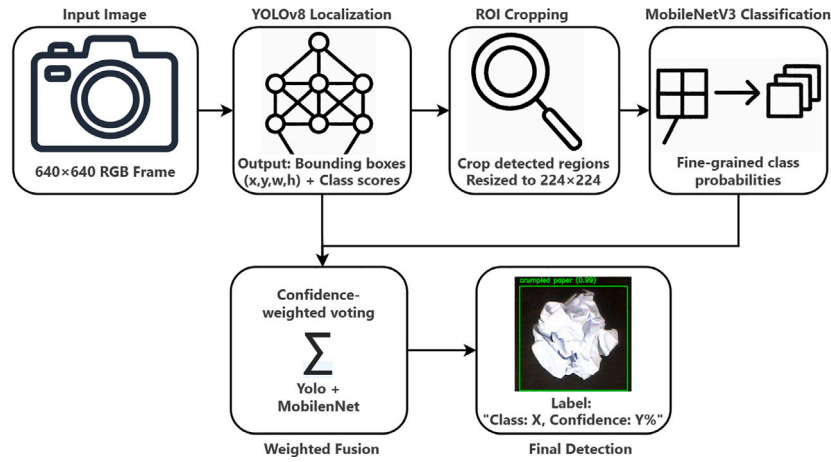


Fig. 3. Confidence-Weighted Voting Mechanism.

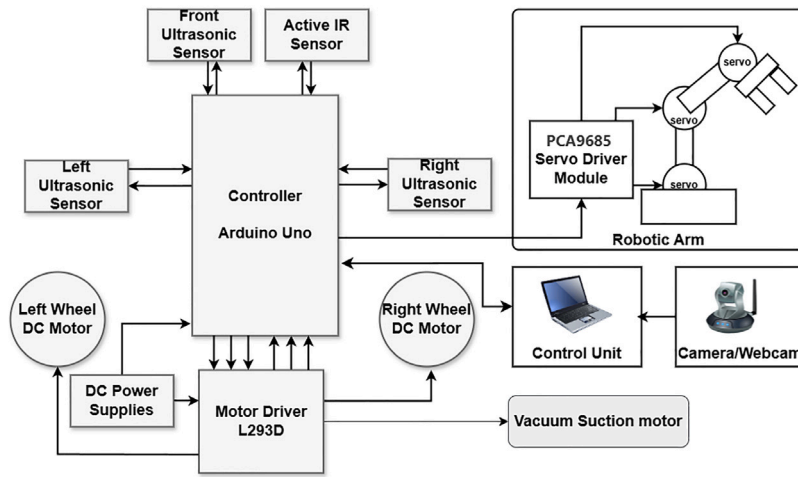


Fig. 4. Hardware System Architecture.

the ideal field of view to detect household objects within its area of operation. The camera sends image data to a detection module of YOLOv8 and MobileNetV3 models working together for enhanced object detection accuracy and precision. The camera resolution was chosen to strike a balance between processing speed and sufficient feature detection precision in a typical home environment.

Actuators

- **DC motors:** For wheel propulsion, the N20 DC gear motors are used, allowing precise control of speeds via PWM signals from the L293D motor driver. They are used to enable the wheels movement for this system and also for the vacuum suction control. These compact and high-torque motors provide omnidirectional movement for the robot's navigation to move forward, backward, or turning, while being power-efficient at the same time.
- **Robotic Arm:** The 5-DOF robot arm was made of lightweight plastic material using 3D printing technology and it uses six servo motors to operate object manipulation tasks. The base and lower joints are supported by three MG995 high-torque servos, whereas the other three SG90 micro servos control the upper arm and claw mechanism, which are generally the lighter parts of this robotic arm. This hierarchical arrangement of motors allow balanced load distribution and energy efficiency while lifting and dropping objects encountered in the cleaning area. The detailed operational

function of object interaction tasks will be discussed in the next section of this paper.

Driver modules

- **PCA9685 servo driver:** This servo driver was used to control the six servo motors present in the robotic arm. This module allows the servos to power independently and hence reduces the load on the onboard voltage regulator of the Arduino. By using I2C communication protocol to connect between PCA9685 and Arduino, it allows the servomotors to connect with its channels rather than to the digital pins of the Arduino, thereby leaving extra pins for other sensors and peripherals if needed.
- **L293D driver:** The L293D dual H-Bridge motor driver controls the N20 DC motors, allowing the robot to control bidirectional movement of its wheels properly through PWM signals from the Arduino. The integrated circuit provides independent control of speed and direction while protecting the microcontroller from voltage spikes due to motors.

3.4.3. Cost analysis and bill of materials

Table 1 summarizes an estimated bill of materials (BOM) to justify the cost-effectiveness of the CleanNav prototype. Prices reflect approximate retail costs in India for readily available, off-the-shelf components used in development. Minor items (e.g., jumper wires, breadboards,

Table 1

Estimated bill of materials (India, approx.).

Component	Description	Estimated Cost (INR)
Arduino Uno	Microcontroller for motor, sensor, and arm control	Rs. 700 – Rs. 1000
HC-SR04 Ultrasonic Sensors (x3)	Obstacle distance measurement	Rs. 60 – Rs. 200 each
IR Edge Sensor	Stair/edge detection	Rs. 30 – Rs. 100
480p USB Camera Module	Vision input for object detection	Rs. 200 – Rs. 500
DC Gear Motors (x2)	Wheel propulsion motors	Rs. 150 – Rs. 300 each
L293D Motor Driver IC	Bidirectional motor control	Rs. 100 – Rs. 150
SG90 Servo Motors (x3)	Light joints of robotic arm	Rs. 90 – Rs. 120 each
MG995 Servo Motors (x3)	High-torque joints of robotic arm	Rs. 250 – Rs. 350 each
PCA9685 Servo Driver	Multi-servo control with external power	Rs. 120 – Rs. 250
LD1117 Voltage Regulator (3.3V)	Voltage regulation for peripherals	Rs. 40 – Rs. 80
3D-Printed Robotic Arm Structure	3D-printed arm links, base, and claw components	Rs. 300 – Rs. 600
Chassis & Mechanical Frame	Wheels, base plate, mounts	Rs. 300 – Rs. 700
Other Components	Jumper wires, switches, connectors, etc.	Rs. 200 – Rs. 400
Total Estimated Cost		Rs. 3000 – Rs. 5,500

connectors, switches, insulation tape, and mounting materials) are combined under “Other Components”. The external computation device used for training and inference is excluded from the estimate.

Commercial robotic vacuum cleaners that offer vision-based navigation and object recognition are typically priced in the range of approximately Rs. 20,000 to Rs. 60,000, with the costs varying according to sensing modalities, onboard processing capabilities, and the overall level of automation that defines the quality. Such systems are primarily designed for navigation and floor-cleaning tasks and generally do not support robotic manipulation or physical interaction with objects in the environment. In comparison, the proposed CleanNav system is constructed using low-cost, off-the-shelf components and open-source software, resulting in an estimated hardware cost of approximately Rs. 3000 to Rs. 5500, while also incorporating a multi-degree-of-freedom robotic arm to enable basic object interaction. Although CleanNav is not intended to directly compete with commercially mature products, particularly in terms of industrial robustness, embedded processing power, and long-term operational reliability, it illustrates that advanced capabilities such as reinforcement learning-based navigation, hybrid deep-learning object detection, and robotic manipulation can be implemented on a substantially lower-cost platform intended for research and educational use.

3.4.4. Communication between python model and Arduino

The serial communication formed between the different modules of the architecture such as the Python-based object detection model and Arduino-controlled hardware system helps in highly effective command transmission, as well as synchronized responses between the systems. The following key interactions were implemented, as summarized in Algorithm 3:

- **Object Detection and Pickup:** When the Python model detects a certain predefined pickup object, commands are sent to the Arduino to stop moving forward, orientate its wheel movements (left, right, forward, or backward) so that the object sits centrally in camera's view, and trigger the robotic arm's pickup sequence through predefined servo angles. Once the object is picked up, the Arduino receives commands to perform next operation explained below.
- **Post-Pickup Actions:** Now the task is simply to drop-off the object carefully. Having picked up an item, the Arduino now receives a command to check for free space available to its left or right with the help of ultrasonic sensors, rotate the robot into the direction of available space, drop the object, turn back to the original orientation, and continue forward to continue with the cleaning activity in the otherwise blocked path.
- **Obstacle Avoidance:** The robot perceives any object not in the PICKUP_OBJECTS list as an obstacle and navigates left or right to avoid it while resuming cleaning work.

- **RL Integration:** For non-pickup objects, a trained RL model takes the decision on the navigation actions, such as moving forward or turning left or right, based on the present state (sensor data and object detection results).
- **Logging and User Interface:** The Python model logs the all recognized items, both non-pickable and pickable objects so that a full detection record is kept for the Lost & Found feature. It also has a simple web interface allowing live video streaming for display and enabling search for previously detected objects using the Lost & Found interface.

Algorithm 3 System Communication and Workflow

```

Input: Sensor data, camera feed.
Output: Robot actions (movement, pickup, drop, navigation).

1: Initialize System:
2: Arduino: Motor control, sensor acquisition, robotic arm.
3: PCA9685: I2C connection, six servo control.
4: Sensors: Ultrasonic (front, left, right), IR (front bottom).
5: RL Model: Load trained RL agent.
6: Main Loop:
7: Sensor Monitoring:
8:   Ultrasonic: readFrontDistance(),
   readLeftDistance(), readRightDistance().
9:   IR: readIR() → HIGH (edge: halt), LOW (continue).
10: Object Detection:
11:   Python: detectObject() → classID.
12:   If classID == pickupObject:
13:     Send command: HALT.
14:     Adjust wheels: centerObject(left/right/forward/backward).
15:     Robotic arm: pickupSequence(servoAngles).
16:   Post-Pickup:
17:     Check space: checkLeftSpace(),
   checkRightSpace().
18:     If left space: turnLeft(), dropObject(), turnRight(), moveForward().
19:     If right space: turnRight(), dropObject(), turnLeft(), moveForward().
20:   Else:
21:     RL Decision:
22:     State: getState(sensorData, objectData).
23:     Action: rlAgent.decideAction(state) → moveForward(), turnLeft(), turnRight().
24:   Repeat:
25:   Continuously: readSensors(), processCameraFeed().
26:   Terminate: If userInput == STOP or taskComplete.

```

- **System Processing and Memory Handling:** All high-level computation such as reinforcement-learning inference, deep-learning detection, and logging, ran on an external control system that was connected to the CleanNav platform via serial communication during prototype testing. This unit executed the Python models in real time, ingested sensor streams, and appended each detection to a locally stored, structured log. The Arduino Uno served strictly

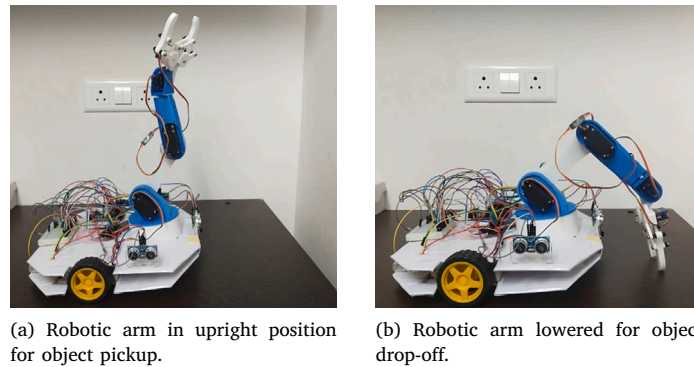


Fig. 5. Robotic arm's movement stages during object handling.

as a low-level interface for motors and sensors and did not store data onboard. This prototype arrangement provided a simple, dependable bridge between hardware and software, enabling stable evaluation of navigation and object-detection performance.

3.5. Robotic arm operation

As already mentioned before, the robotic arm of the autonomous vacuum cleaner is utilized to move away specific objects that might interfere with the cleaning path. The arm movements are predefined, and ensures precise object pickup without engaging complex computations. The following subsections explain the arm design, the object pickup strategy, and the item drop-off process.

3.5.1. Robotic arm design and hardware setup

The robotic arm has six servo motors that are committed to particular movements such as base rotation, shoulder, arm extension, and action of the claw. The claw can completely open or close to grasp objects firmly. The arm has pre-programmed motion sequence to enable repeatable and consistent pickup and drop-off operations on items.

The motor wheels of the robotic cleaner, driven by an Arduino Uno, play a crucial role in the robot's positioning to enable objects that are detected to be positioned in the center of the camera view to facilitate accurate arm movements towards the detected objects which needs to be picked based on the predefined list of pick-up items. Only if the object to be picked is at the center of the camera's view, the pickup process will be initiated. Fig. 5 shows the robotic arm's movement stages during object handling.

3.5.2. Object pickup strategy

The object pick-up action is controlled by a combination of object detection models and predefined arm motion sequences:

Identifying Pickup Objects: It detects objects using YOLOv8 and MobileNetV3. Once it has identified an object from the given PICKUP_OBJECTS list,

{cell phone, spectacles, crumpled paper, mouse, remote, bottle, socks}

the pickup process is initiated in the system. Other additional items may be added in this list based on environmental factors and cleaning requirements while also considering the holding capacity of the claw.

Positioning the Arm for Pickup: The robot's wheels are actively steered so that the vacuum cleaner will travel left, right, forward, or backward until the object detected is in the center the camera's view. This dynamic positioning method ensures that the object is aligned correctly with the pre-programmed motion of the robotic arm, thus reducing the incidence of missed pickups. As soon as the object is centered to the view, the robotic arm proceeds to perform a sequence to lower its arm, grab the object firmly with the claw, and lift up.

3.5.3. Object drop-off strategy

After it has successfully lifted an object, the robot evaluates the surroundings by checking left and right to determine the side with more available space, with the help of ultrasonic distances. It then turns towards that side, drops its arm, opens its claw completely to release the object, and returns to its original forward-facing position. This sequence of movement allows the robot to keep moving forward constantly. As the robot moves forward, the vacuum suction system at the bottom remains active to allow for effective cleaning of the floor in the newly cleared area. This integrated approach improves the efficiency of the robot in cleaning by eliminating possible obstructions that would otherwise hinder coverage.

Working together with the other modules of the system, especially the cooperative sequence between the RL navigation model and arm control ensures that decision-making and manipulation remain synchronized throughout operation.

3.6. Lost & found system

Lost & Found System was established to enable individuals to track their lost items in a given household setting. The system uses object recognition model, specifically the combined approach of YOLOv8 and MobileNetV3, to store lists of detected items and respective contextual information. It enables the users to get clues on where the lost item was last detected, particularly by providing details of nearby detected items so that they get idea on where to search in the environment.

3.6.1. Detection logging mechanism

To create an effective tracking system, a logging module was introduced to keep a sequence of records for identified objects. A record for each recognized object will consists of the following data which are useful for retrieval approach:

- **Timestamp** — The specific date and time the object was seen.
- **Detected Item** — The object label identified based on combined decisions from YOLOv8 and MobileNetV3 detection.
- **Confidence Score** — The confidence level of the system regarding the detected object to give a score of how sure it is about the detection, in order to ensure reliability.
- **Nearby Objects** — To enhance contextual understanding of the environment and the obstacles or items present, the system captures data regarding objects identified in close proximity to the target object. This offers valuable insights into the immediate environment.

By integrating this information into a logically structured log file, the system maintains a vast repository of detections. This feature enables users to ask the system for specific items and receive relevant information about their latest sighting.

Table 2
Comparison of YOLOv8 and YOLOv11 metrics.

Metric	YOLOv11	YOLOv8	Superior model
Precision	0.8351	0.8908	YOLOv8
Recall	0.7321	0.7599	YOLOv8
F1-Score	0.7376	0.7809	YOLOv8
mAP@50	0.8353	0.8029	YOLOv11
mAP@50-95	0.6841	0.6434	YOLOv11
FPS	29.36	55.52	YOLOv8

3.6.2. Tracking object locations

The system does not use active tracking techniques; instead, it uses the detection log to deduce the last known location of an object. When a user requests a search for a lost item, the system fetches the latest log entry for the object. Apart from the timestamp, objects around the location are shown to provide additional clues on the possible location of the item. For example, if the log shows that “Keys” had been detected earlier around a “pen” and “TV”, the user is prompted to search around the general area. This approach gives good assistance to the user without having to employ the robotic cleaner to participate in active navigation in pursuit of the item.

3.6.3. Search and retrieval process

The search interface allows the user to engage with the system by inputting the object’s name sought. On entry, the system gives feedback with the following information:

- The timestamp the object when it was last detected.
- The surrounding objects which were also detected during the latest detection of target object.

This optimized process provides users with helpful clues to enable the recovery of lost items. The system operates in the background of regular cleaning operations, thus having a minimal impact on system performance overall while still providing helpful information for item retrieval. With the use of object detection information and environmental clues, the lost & found System can assist users in item location without actively involving the robotic movement for the sole purpose of that.

4. Results and discussion

This section evaluates the system’s performance by comprehensively looking into different perspectives like navigation performance, accuracy of object detection, and operation success of the robotic arm. In this section, the detailed evaluation results of the system are presented, emphasizing the detection performance of the YOLOv8 model, the classification accuracy of the MobileNetV3 model, and overall reliability in real-time environments.

4.1. Model comparison: YOLOv8 vs. YOLOv11

A very insightful evaluation was conducted for both YOLOv8 and YOLOv11, in order to select the best object detection model. The evaluation metrics included precision, recall, F1-score, mAP@50, mAP@50-95, and inference speed (FPS). A summary of results is contained in Table 2 and Fig. 6.

The mAP@50 or mAP@50-95 values listed in Table 2 derive from evaluating trained models on a separate test images folder. This evaluation reflects the real-world detection performance of the models.

YOLOv8 performed better than YOLOv11 in terms of precision, recall, and F1-score, reflecting higher overall detection accuracy. YOLOv8 also showed much greater inference speed (55.52 FPS) than YOLOv11 (29.36 FPS), making it more applicable for real-time use. Although YOLOv11 showed slightly greater mAP@50 and mAP@50-95, the

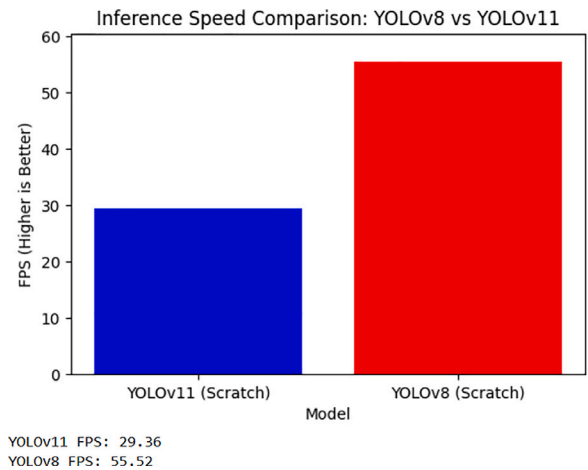


Fig. 6. Inference Speed Comparison (YOLOv8 vs. YOLOv11)

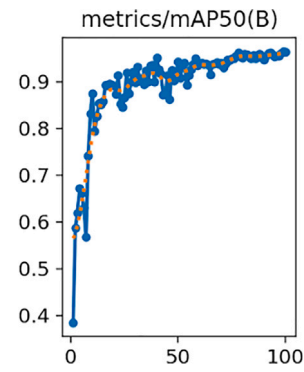


Fig. 7. mAP@50 curve across training epochs.

higher speed and detection accuracy of YOLOv8 made it the best option for this system, where real-time performance will require all the computations to be very fast.

This performance variation is due to the improvements in YOLOv8 architecture. YOLOv8 takes advantage of the CSPDarkNet backbone and path aggregation network (PAN) to improve feature extraction with less computational load. YOLOv11, however, employs improved building blocks including C3K2 blocks, Spatial Pyramid Pooling Fast (SPPF), and Cross-Stage Partial Self-Attention (C2PSA) to enhance small object detection and aggregate accuracy. However, these improvements impose overhead complexity, therefore, potentially increasing the inference time compared to that of YOLOv8. The outcome would be that YOLOv11, although more recently introduced model, gives a slight compromise in speed in favor of improved feature representation, while YOLOv8 maintained a fine balance between real-time performance and accuracy.

4.2. YOLOv8 model evaluation

The performance parameters studied for evaluating the YOLOv8 model include commonly used standard evaluation metrics such as mean Average Precision (mAP) and the Precision-Recall (PR) curve.

As presented in Fig. 7, the mAP@50 curve serves as an illustration of how the performance of the model has evolved during the process of training through different epochs. The curve exhibits a quick rise in performance during the initial epochs, followed by a more moderate increase to converge toward an mAP@50 of approximately 0.96, which indicates efficient learning and, thus, ensures stability of the model in terms of detection capability in subsequent iterations.

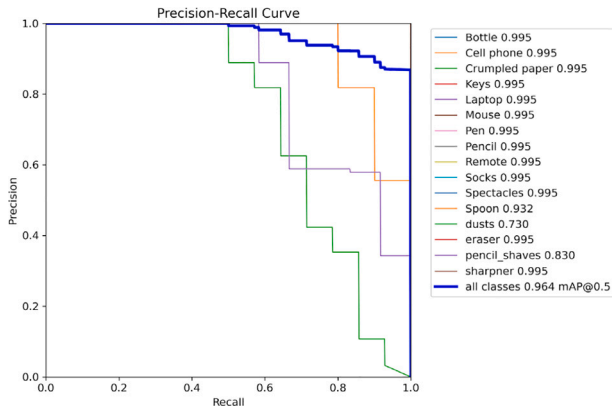


Fig. 8. Precision-Recall curves for different object categories.

It should be noted that the performance improvement shown by mAP@50 value of 0.96, as demonstrated in Fig. 7 shows how the model improved over epochs on the training dataset, which is entirely different from the test evaluation in Table 2, where an mAP@50 of 0.8029 was obtained on real-world test images by YOLOv8.

Fig. 8 the Precision-Recall (PR) curves for individual object categories to probe the class-wise detection performance of the model. Most of the object classes are performing at high precision and recall, with some of them close to being perfectly detected. Some objects, such as “bottle”, “cell phone”, “keys”, “pencil”, and “spectacles”, yielded very high precision of 0.995, thus indicating that the model is almost certain about detecting these objects.

However, certain categories like “dusts” (0.730) and “pencil_shaves” (0.830) have comparatively lower precision scores. These results might suggest potential challenges in visually similar item discrimination, suggesting some areas of improvement in classification. Despite these slight variations, the overall average mAP@50 for all categories is calculated to be 0.964, which reaffirms the model’s overall competent detection ability to be excellent.

These results prove that the YOLOv8 model has been very useful in detecting a wide variety of objects with reasonable precision and recall. Future improvements could address fine-tuning the detections for specific objects and enhancing the feature representations for those ambiguous categories.

4.3. MobileNetV3 performance analysis

MobileNetV3 had performed excellently in classifying 16 object categories relevant to the system’s object detections and interaction tasks, boasting an overall accuracy of 99.20% which indicates the fine discrimination capability of the model.

4.3.1. Confusion matrix evaluation

The trained mobilenetv3 model performance can be described in terms of the confusion matrix as shown in Fig. 9, which explains the models prediction accuracy over the 16 classes of items.

Most categories have higher accuracies with only some of the classes having few misclassifications. The classifications are excellent for some of the challenging and small object types such as the “pencil shavings”, “keys”, and “spectacles” which are pertinent to the robotic cleaner’s tasks. Visually similar items, such as “Crumpled paper” and “Socks” if they appear in same shape and color, had few misclassification occurrences observed.

Thus, the MobileNetV3 results show the object identification capabilities, which lead to increased precision in handling objects for robotic-arm operations in the system.

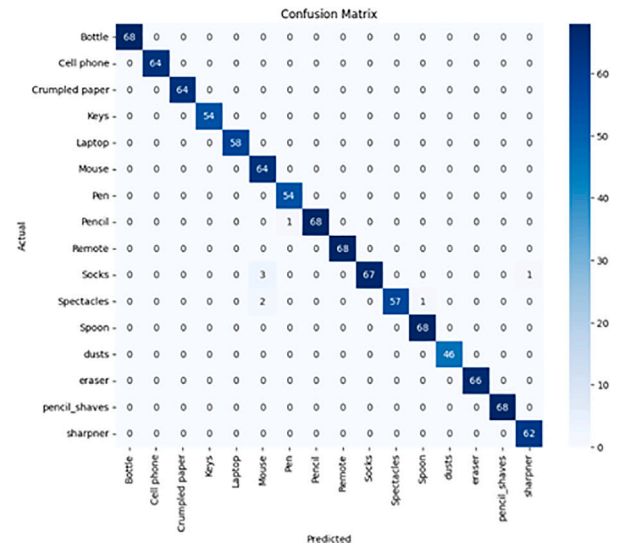


Fig. 9. Confusion Matrix showcasing MobileNetV3’s classification performance across the 16 object categories.

4.4. Combined detection module via weighted voting

In order to further improve object detection accuracy, the prediction results as outputs from the YOLOv8 detection model and MobileNetV3 classification model are combined together through weighted voting to produce a more reliable decision. This technique, therefore, combines the advantages of both models — YOLOv8 for localization, and MobileNetV3 for classification.

As illustrated in Fig. 10, the decisions in the weighted voting approach will be helpful in the event that different labels were chosen by YOLOv8 and MobileNetV3. Hence, the final label in this case would depend on the confidence scores of both models. The model with the higher confidence is given greater weight in the final decision, thus allowing the object detection module to choose the correct classification label more reliably.

Numerous objects were tested in order to validate the effectiveness of this method of fusion, and findings are noted. Results of a comparative study of different predictions by YOLOv8 and MobileNetV3 alongside the final verdict made after weighted voting is noted in Table 3. The results show that weighted voting mechanism rectifies the wrong classifications made by either of the two models, especially when it comes to smaller objects in our dataset such as “Pencil” and “Keys”.

The table also shows instances where YOLOv8 went wrong in classifying objects like “Pen” instead of “Pencil” or “Remote” instead of “Cell phone”, but got corrected by the integration with MobileNetV3. This corroborates the effectiveness of model-fusion for reducing false positives and enhancing classification accuracy.

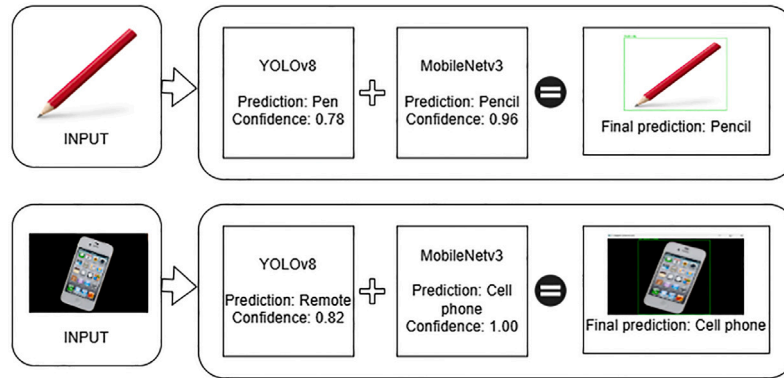
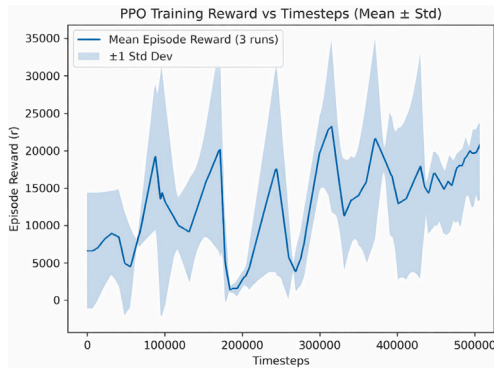
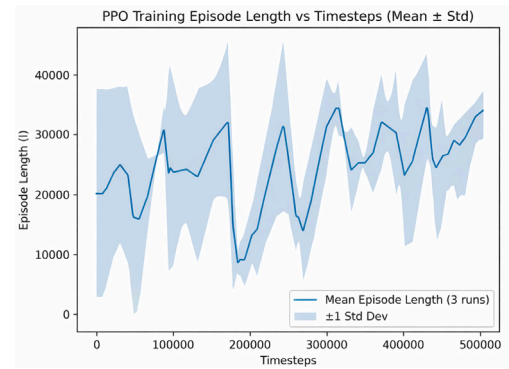
These results validate the performance advantage of a hybrid detection-classification pipeline wherein object localization and fine-grained classification jointly interface for enhanced efficacy in performance.

The hybridization of bounding box predictions from YOLOv8 and classification confidence from MobileNetV3 gives the system a highly improved and very accurate object recognition capability. This tactic becomes especially applicable in a real-time robotics scenario where speed and decision-making rely on extremely accurate input. Future optimizations could include adaptive weight strategies that will automatically tune model input based on object complexity and environmental conditions.

Table 3

Comparative analysis of weighted voting results.

Actual object	YOLOv8 prediction	MobileNetV3 prediction	Weighted voting prediction
Pencil	Pen (0.78)	Pencil (1.00)	Pencil
Cell phone	Remote (0.82)	Cell phone (1.00)	Cell phone
Crumpled paper	Crumpled paper (0.90)	Crumpled paper (1.00)	Crumpled paper
Keys	Spoon (0.79)	Keys (0.95)	Keys
Mouse	Mouse (0.87)	Mouse (1.00)	Mouse
Pen	Pen (0.82)	Pen (1.00)	Pen
Remote	Remote (0.92)	Remote (0.99)	Remote
Spoon	Spoon (0.91)	Spoon (1.00)	Spoon
Socks	Socks (0.81)	Socks (1.00)	Socks
Socks	Bottle (0.53)	Socks (1.00)	Socks

**Fig. 10.** Illustration of Weighted Voting Process in Object Detection module.**Fig. 11.** Episode Reward Mean $\pm$ Standard Deviation Across Training Runs.**Fig. 12.** Episode Length Mean $\pm$ Standard Deviation Across Training Runs.

4.5. Reinforcement learning-based navigation performance

The evaluation of the reinforcement learning (RL) model's performance was carried out through key metrics and visual representation of the training process. The results emphasize the strengths of the model in navigation, strategizing an efficient path, and avoidance of collision.

4.5.1. Training insights and observations

The training process revealed several meaningful trends. The Episode Reward mean as shown in Fig. 11 experienced an overall upward shift with episodic fluctuations, which indicates improved decision-making capabilities and effective reward acquisition. Such consistency of increment showcases the model's improved movement planning and learning stability throughout the training.

Likewise, the Episode Length Mean graph showed increased long-term episode lengths as displayed in Fig. 12, which implied better obstacle avoidance and extended navigation capability. This aligns with the model's successful navigation without early stopping.

The shaded region in Figs. 11 and 12 represents ± 1 standard deviation across three independent training runs, providing insight into training stability.

To further give an insight of the learning process, Fig. 13 shows the trajectory evolution of the model across several stages of training. The first two subfigures ((a) and (b)) correspond to earlier models trained for 250,000 timesteps, at a low learning rate, where we observe limited exploration of the working environment and frequent collision with obstacles. The movement patterns tend to resemble a bounded path indicating that the model had not properly learnt effective means of navigation.

In contrast, the next three subfigures ((c), (d), and (e)) depict the trajectory of the final model after training for 500,000 timesteps under PPO hyperparameters — a learning rate of 0.003, an entropy coefficient of 0.01, and an episode length of 40,000 steps. This trajectory shows much more exploratory behavior, covering larger areas while maintaining mostly successful obstacle avoidance. However, some runs

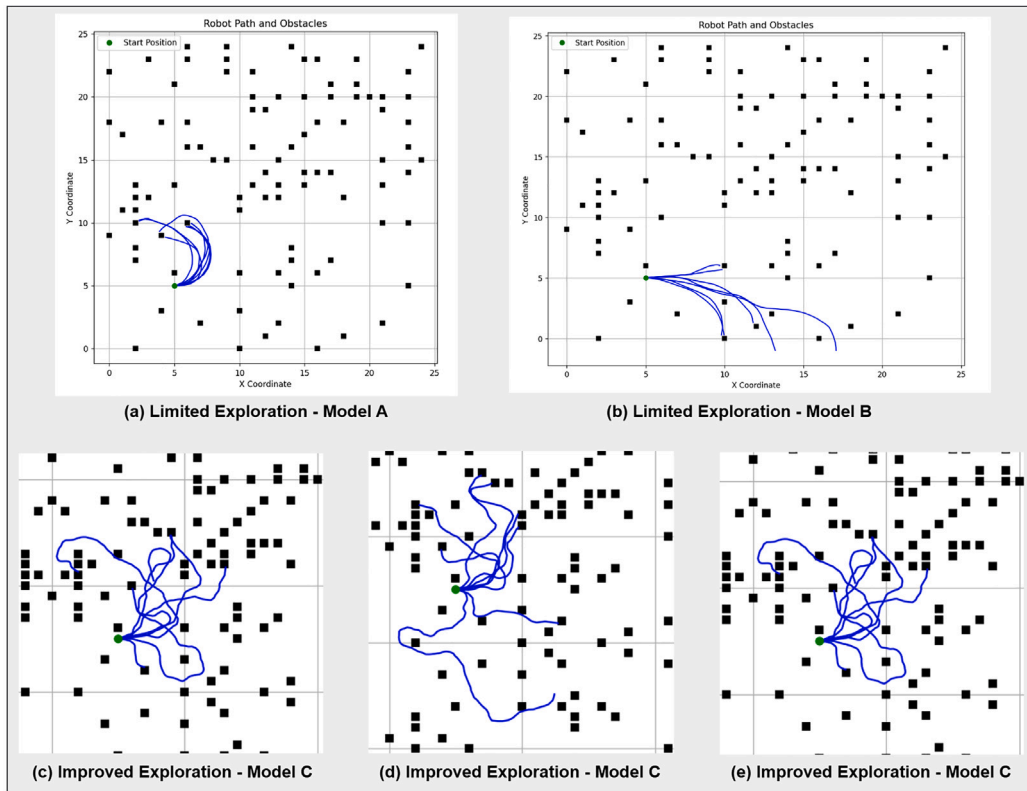


Fig. 13. RL Model Trajectory Evolution Across Multiple Training Stages.

Table 4

RL model performance metrics.

Metric	Value	Observation
Average Coverage	60.44%	The model achieved reasonable area coverage, but there is scope for improved exploration strategies.
Average Steps per Episode	22,222.22	Prolonged episode lengths suggest the model effectively avoided obstacles, enabling extended exploration.
Total Collisions	6	The number of collisions is not significantly high. These collisions could be further reduced through direct command overrides based on ultrasonic sensor data during hardware deployment.
Average Reward	7204.00	The positive rewards indicate the model has succeeded in maximizing coverage while avoiding obstacles, which means the model is learning correctly.

did collide with obstacles, and others ended its path in open space suddenly due to reaching the maximum episode length — both scenarios indicating refined exploration and avoidance for longer periods.

This transition from constricted movement to extensive exploration, along with occasional collisions, marks a step progress in the RL-model learning. Moreover, the final model behavior is coherent with the desired Episode Reward Mean and the Episode Length Mean trends, which supports its optimization of navigation strategies. In terms of hardware deployment, direct overrides based on ultrasonic sensor data will ensure an extra layer of reliability by preventing collisions during execution in the real world.

4.5.2. Navigation metrics summary

The RL model's navigation performance measures were analyzed by observations on aspects such as path exploration, obstacle avoidance, and coverage maximization. The results, as shown in Table 4, indicate

that the model effectively performs conflict resolution between prolonged navigation and collision avoidance while ensuring maximal area coverage.

4.5.3. Comparative analysis with classical navigation baselines

In classical and heuristic Coverage Path Planning (CPP) for ground-based indoor robots, performance is typically judged based on the percentage of area covered, alongside travel time, path overlap, and energy use. Drawing on a large survey as shown in Table 5, Tan et al. [11] report coverage results ranging from 76% for a discrete Invasive Weed Optimization (IWO) approach up to 97.98% with a Grey Wolf Optimizer (GWO) approach on mapped environments with obstacles. Between those extremes, a Genetic Algorithm (GA) + Neural Network hybrid achieves 93.74% coverage with 4.25% overlap, and a modified A* planner yields a 6.4% coverage gain over the traditional A* baseline. All of these results come from trials in 2D, floor-like settings, the same kind of environment a vacuum-style robot operates in. SLAM-based systems typically perform best when the trajectory is planned.

Table 5
Comparative performance of CleanNav against classical and heuristic baselines.

Baseline family	Reported result	Notes
Modified A*	+6.4% coverage over traditional A*	Heuristic improvement, 2D ground simulation
GA + Neural Net	93.74% coverage, 4.25% overlap	Single-robot, 2D grid environment
GWO (Multi-Robot)	97.98% average coverage (4 obstacle maps)	High upper bound; multi-robot
IWO Variant	76% maximum coverage	Lower-bound classical baseline
SLAM + B-A* (Zhao & Hwang, 2024)	95%–98% coverage	Structured, map-based; ROS + SLAM integration
CleanNav (this work)	~60% coverage	Dynamic, map-free, single robot; emphasizes adaptability

In an 8×8 m indoor experiment, Zhao and Hwang [12] achieved a 95%–98% coverage ratio using a ROS-integrated SLAM system with Bidirectional A* (B-A*).

All these levels of performance, however, is achieved mostly in a structured or mapped environment with reliable sensor-based localization, showing a dependence on environmental stability. In this context, CleanNav's ~60% coverage in dynamic, unmapped environment shows a clear advantage, whereby our system gives up some area coverage in exchange for flexibility and real-time adaptability.

We compare against survey-reported baselines that explicitly quantify coverage in comparable 2D indoor settings. Although CleanNav's numerical coverage is lower, it achieves robust navigation without pre-mapping and remains effective under dynamic environmental changes, which classical methods generally cannot accommodate.

4.5.4. Hardware deployment observations of the RL model

The RL model was deployed on the robotic cleaner's hardware system. Ultrasonic sensor inputs were fed to the trained model as inputs, which made decisions based on its policies acquired through the learning process during its training. Although the model navigated properly most of the time, there were minor collisions at certain scenarios. These can be avoided by direct overrides from ultrasonic sensor data whereby if the obstacle is close enough, the robot can be made to turn instead of waiting for the RL model's decision.

In short, the reinforcement learning approach worked well to enhance the vacuum cleaner's navigation capability with proper obstacle avoidance, sufficient area coverage, and steady learning behavior. Further enhancements to the reward tuning and exploration strategy may possibly drive its overall performance even higher.

4.6. Robotic arm and base system performance

It should be noted that the proposed robotic arm is designed as a lightweight, low-cost manipulator integrated with a mobile vacuum platform for domestic environments. In contrast to industrial stationary manipulators such as Baxter, built for high-precision, high-payload operation in structured settings, our system prioritizes affordability, mobility, and reliable behavior in dynamic home scenarios. Accordingly, our evaluation emphasizes task feasibility and practical reliability rather than direct performance comparison with industrial robotic arms.

Energy considerations. During the manipulation experiments, the arm does not operate continuously; it is triggered mainly during pickup attempts and drop-off actions. As a result, its energy use occurs in short bursts rather than as a constant load, while the base platform continues navigation and cleaning. We also observed that heavier objects increased mechanical load and servo strain (including one damage case), suggesting that object weight and gripper geometry are major factors influencing actuation effort. For perception, the system uses a 480p camera stream to balance runtime efficiency with reliable detection. Overall, the evaluation focuses on feasibility under realistic constraints, with a complete battery runtime characterization is left for future work when the system is deployed with onboard compute and a defined battery pack.

In order to compare the combined functionality of the robotic arm's object handling capabilities with the base positioning system of the

vacuum cleaner, the following measurements and observations are outlined:

4.6.1. Object pickup analysis

The robotic arm's pick-up performance varied considerably with respect to object size, shape, and weight. Some important observations were made as follows:

- **Light and Flexible Objects:** Objects like crumpled paper and socks usually had a suitable shape and weight for grasping of the claw, leading to successful pick-up by the robotic arm. These objects achieved around a 90% success rate in pickup attempts.
- **Heavy or Rigid Objects:** Few heavy objects such as bottles and mobile phones were more complicated. Although, the robotic arm tried to grasp them, they were too heavy for the servo motors and in one instance damaged one of those actuators handling the claw of the robotic arm.
- **Objects with Irregular Shapes:** Spectacles, for instance, proved difficult to hold securely because they were irregularly shaped and so did not lend themselves well to securing by the claw.

The summary given in Table 6 shows high success rates for light items and although some objects were challenging for robotic arm task, they were still logged successfully for the lost & found feature in the background.

4.6.2. Quantitative manipulation success rate analysis

A quantitative evaluation of the robotic arm was carried out through repeated pickup trials on common household objects typically encountered during cleaning tasks. The evaluation was limited to pickup-eligible objects, selected in accordance with the mechanical limitations of the lightweight two-claw gripper and the available servo torque. To better reflect differences in grasp stability and load characteristics, the objects were grouped into *lightweight/regular-shaped* and *heavy/irregular-shaped* categories. Each object was tested over 20 pickup attempts under identical operating conditions (see Table 7).

In these trials, lightweight, regular-shaped objects were handled reliably, achieving a 90% pickup success rate. This level of performance is consistent with the claw geometry and the relatively low torque required to lift and hold these items. Performance dropped markedly for heavier or irregularly shaped objects, with a success rate of about 23%. The main reasons were limited servo torque, the lightweight two-claw gripper design, and the lack of adaptive grasping or force-feedback, which made stable enclosure harder to achieve. Items with uneven geometry or higher mass tended to slip or to be only partially enclosed by the claw during grasp attempts.

Taken together, these results match the system's intended role of moving lightweight obstacles in a home for cleaning to continue, rather than performing general-purpose manipulation. For this reason, widely used benchmarks such as the Cornell and Jacquard datasets, built around learning-based grasp prediction, do not align well with evaluating this rule-based, task-specific manipulation strategy.

4.6.3. Object centering accuracy

The base of the vacuum cleaner was tasked with centering objects within the camera's view to facilitate the pickup operation as the

Table 6

Robotic arm and base performance summary.

Feature/Observation	Achieved	Notes/Observations
Successfully picked up crumpled paper	Yes	90% success rate; lightweight and flexible.
Successfully picked up socks	Yes	90% success rate; lightweight and flexible.
Successfully picked up spectacles	No	Irregular shape made gripping difficult.
Successfully picked up cell phone	No	Heavy object; strained servo motor; logged instead.
Successfully picked up bottle	No	Heavy object; servo strain caused motor damage.
Successfully logged cell phone (Lost & Found)	Yes	Detected and logged despite failed pickup attempts.
Successfully logged bottle (Lost & Found)	Yes	Detected and logged despite failed pickup attempts.
Object centering in camera's view	Partial	Occasional misalignment between camera's center view and object position.
Correct centering, robotic arm failure to grasp object	Yes	Due to servo wear or limitations of the claw or the shape of the object.

Table 7

Robotic arm pickup performance by object category.

Object category	Object included	Trials per object	Successful pickups	Mean success rate (category)
Lightweight/Regularly-shaped Objects	Socks	20	18	90%
	Crumpled Paper	20	19	
	Empty Plastic Bottle	20	17	
Heavy/Irregular-shaped Objects	Remote Control	20	6	~23%
	Cell Phone	20	0	
	Mouse	20	7	
	Spectacles	20	5	

robotic arm was predefined to move towards the spot falling to the center of camera's field of view. The centering mechanism tended to work successfully most of the time, which could be attributed to the camera's narrow field of view. This small viewing angle operated as if the view were a zoomed-in shot, so centering objects was simple. As the view from the camera was focused to a smaller region and not wide angles, variables such as object depth did not have much impact on alignment and the robotic arm was able to reach the object's position in most of the attempts. So long as the object looked centered, the robotic arm would typically place itself in the proper position and make contact with the object.

Even with successful positioning of the objects in camera's view, pick-up efforts failed at times due to:

- **Claw Design Limitations:** The robotic arm claw, with its two-finger grip, found it difficult to grasp thin or irregularly shaped objects such as spectacles.
- **Servo Strain and Wear:** The servo motors are overstrained with repetitive attempts in holding heavy objects which caused them damage.

These results are analyzed through different performance measures as reflected in [Table 8](#).

4.6.4. Statistical validation of robotic arm performance

To evaluate reliability and consistency in the arm's object-manipulation task, 95% confidence intervals together with variance and standard error was analyzed. For lightweight objects, the pickup rate was 90% with a tight interval [0.596, 0.982], consistent with stable, repeatable performance. Whereas the overall success rate had wider bounds [0.388, 0.837], and failures associated with claw/servo limitations lay within [0.117, 0.547], implying greater uncertainty when the robotic arm works with heavier or irregularly shaped objects, which is due to its light-weight two-finger gripper claw. The Lost & Found feature recorded every detectable object (100%) with an interval of [0.785, 1.000], showing that background detection remained dependable even when a physical grasp failed.

Note: Confidence intervals were calculated using the Wilson score method at the 95% level ($z = 1.96$), which provides accurate bounds for small sample sizes and proportions close to 0 or 1.

To further examine variation, [Table 9](#) summarizes the estimated variance and standard error for the pickup success rates. For light objects, the variance was 0.009 and the standard error 0.095, suggesting

that the success rate may vary by roughly $\pm 9.5\%$ across repeated trials. This small deviation demonstrates consistent grasping for light, flexible objects. For all objects combined, the variance increased to 0.0164 and the standard error to 0.128, reflecting moderately higher variability when heavier or irregularly shaped items were tested.

Low variance for lightweight objects indicates that the arm operates steadily and achieves reliable grasps across repeated trials. The modest rise in variability for the full set of objects reflects practical constraints of the gripper's geometry and available servo torque when dealing with irregularly-shaped or heavier items. Taken together, the 95% Wilson confidence intervals ([Table 8](#)) and the variance results ([Table 9](#)) provide converging evidence that the arm's object-handling performance is statistically reliable and consistent, particularly for lighter objects.

In low-cost domestic service robotics, mobile platforms equipped with onboard robotic arms or grasping mechanisms are still relatively rare. Most existing vacuum and floor-cleaning robots in this category focus primarily on navigation, obstacle avoidance, and cleaning efficiency, without attempting physical manipulation. Consequently, direct grasping benchmarks for systems operating under similar cost and deployment constraints are limited. Within this context, the proposed robotic arm is evaluated in terms of task feasibility, consistency across repeated trials, and well-defined mechanical limitations, rather than through direct cross-platform manipulation benchmarks.

4.6.5. Robotic arm performance summary

The robotic arm could pick up and hold lighter objects such as crumpled paper and socks without difficulty. However, when it came to heavier or irregularly shaped objects like cell phones or bottles, the robotic arm could not execute its pickup and drop-off operations owing to its physical design and mechanics, particularly the shape of its claw and its own light weight posing inability to lift heavy loads. In all these situations, the lost & found system continued its function in the background by logging the information of all the detected objects for retrieval or human intervention when necessary. A combination of observations referred to in [Tables 6, 8](#) and [9](#) demonstrates the strength of the system in handling certain types of objects and points out the limitation that arises from errors in object alignment, design constraints in the claw, and stress placed on servo motors.

Overall, the robotic arm in this system is not intended to achieve industrial-grade precision grasping, instead, it is designed to relocate small obstacles in typical household settings so the robot can continue cleaning without interruption.

Table 8

Quantitative performance metrics with 95% confidence intervals (Wilson score method).

Metric	Value	95% confidence interval (Wilson)
Pickup success rate (light objects)	90%	[0.596, 0.982]
Total pickup attempts	14	–
Successful pickups (overall)	9	[0.388, 0.837]
Failed pickups (due to object misalignment)	2	[0.040, 0.399]
Failed pickups (due to claw/servo issues)	4	[0.117, 0.547]
Logged objects via Lost & Found system	100%	[0.785, 1.000]

Table 9

Estimated variance and standard error for pickup success rates.

Category	Success rate	Variance	Standard error
Light objects	90%	0.009	0.095
All objects (overall)	64.3%	0.0164	0.128

5. Future work and system enhancements

The current CleanNav prototype delivers reliable object recognition and lightweight manipulation, but several mechanical factors limit its scalability and robustness. The two-finger gripper claw offers few contact points and insufficient lateral stability, reducing its grasping capability for irregularly shaped objects or heavier items. Although the 3D-printed frame supports rapid iteration and low mass of the hardware system, it lacks the stiffness or rigidity required to hold onto heavy load of items. The use of standard micro-servos, which are commonly applied in small IoT applications and experiments, constrains performance and their limited torque contributes to slippage and incomplete lifts while trying to perform its object manipulation tasks on heavier items. Future iterations should consider grippers with wider claws, multi-finger configurations, or high-friction surfaces of those grippers to improve stability while lifting or holding the obstacle items. Additionally, replacement of the current servo motors with higher torque servo actuators and lightweight metallic joints to build the robotic arm could increase the payload capacity of the arm, with reduced mechanical strain and enhanced durability.

The Lost & Found module currently records all the detections consistently and supports reliable retrieval through the web interface as long as the object has been identified through the camera. However, to properly understand its value in everyday use and also get ideas on further enhancements to eliminate any irregularities or edge cases, a small, formative user study would be appropriate. This evaluation would solicit feedback on how well the module's contextual clues, such as timestamps and indicators of nearby objects, would help users recover misplaced items. Although the feature is fully automated and has been functionally validated in software in this CleanNav system, user-centred testing would provide additional evidence of its practical effectiveness and guide targeted design refinements.

6. Conclusion

In this work, an intelligent cleaning system, the CleanNav, was developed by combining different smart modules which are reinforcement learning-based navigation, robotic manipulation, and deep learning-based object detection to build the novelty of this work. The reinforcement learning model displayed consistent learning behavior, reasonably covering areas with a fairly low count of collisions. Minor collisions realized during the deployment were manageable through overrides of RL model's decisions with direct commands based on ultrasonic sensor data, for more reliability in real-world conditions. Object localization and classification features, using YOLOv8 and MobileNetV3 models, significantly improved the system's ability to identify and interact with a variety of objects in its environment. This integration was of immense utility in solving the problem of misplaced items, as evidenced by the lost & found system that logged details of all the detected objects.

For object manipulation, although lightweight objects such as crumpled paper and socks were grasped successfully by the robotic arm, heavier or irregularly shaped objects posed a challenge as the claw was not able to get a good grip. Nevertheless, despite these challenges, the system has performed reliably in orienting objects for pickup, while also letting the system log the times when it failed the attempt to pick an object. Overall, the system exhibited itself as a promising autonomous robotic cleaning solution, while also leaving some room for enhancement in reward tuning and exploration strategies, as well as in the direction of hardware optimization for longevity and extended object handling capability. The main contribution of this work lies in the system-level integration of PPO-based reinforcement learning, real-time robotic manipulation, and hybrid vision modules into a unified cleaning platform. This integration between perception, navigation, and actuation advances beyond just the traditional detection-focused solutions, demonstrating a practical foundation for adaptive and interactive home robotics, the future of current technology.

CRedit authorship contribution statement

K. Hemalatha: Supervision, Investigation, Conceptualization. **R. Tshewang:** Methodology. **V. Tapaswi:** Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors sincerely thank VIT Chennai for financial support and institutional facilities provided to carry out this research work.

Data availability

No data was used for the research described in the article.

References

- [1] Radha KK, Chakravarthy MA. Fabrication of solar robotic vacuum cleaner. *Mater Today: Proc* 2023. <http://dx.doi.org/10.1016/j.matpr.2023.06.401>.
- [2] Hwang W, Kim J, Han J, Kang W, Park S. Investigation of resonance avoidance method for vacuum cleaner. *IEEE Open J Ind Appl* 2023;4:328–35. <http://dx.doi.org/10.1109/OJIA.2023.3323343>.
- [3] Wang H, Liu C, Cai Y, Chen L, Li Y. YOLOv8-QSD: An improved small object detection algorithm for autonomous vehicles based on YOLOv8. *IEEE Trans Instrum Meas* 2024;73:2513916. <http://dx.doi.org/10.1109/TIM.2024.3379090>.
- [4] Wang S, Wang Y, Zheng B, Cheng J, Su Y, Dai Y. Intrusion detection system for vehicular networks based on MobileNetV3. *IEEE Access* 2024;12:106285–302. <http://dx.doi.org/10.1109/ACCESS.2024.3437416>.
- [5] Safaldin M, Zaghdien N, Mejdoub M. An improved YOLOv8 to detect moving objects. *IEEE Access* 2024;12. <http://dx.doi.org/10.1109/ACCESS.2024.3393835>.
- [6] Tian X, Shi L, Luo Y, Zhang X. Garbage classification algorithm based on improved MobileNetV3. *IEEE Access* 2024;12:44799–807. <http://dx.doi.org/10.1109/ACCESS.2024.3381533>.
- [7] Moon W, Kim T, Park B, Har D, Nengroo SH. Path planning of cleaning robot with reinforcement learning. 2022, arXiv preprint [arXiv:2208.08211](https://arxiv.org/abs/2208.08211).

- [8] AM, Oliver CA, Joseph AM. Robot vacuum cleaner using SLAM and ROS. In: 2024 international conference on knowledge engineering and communication systems. 2024, p. 1–6. <http://dx.doi.org/10.1109/ICKECS61492.2024.10616915>.
- [9] Pacal I, Akhan O, Deveci RT, Deveci M. An efficient real-time colonic polyp detection with YOLO algorithms trained using negative samples and large datasets. *Comput Biol Med* 2022;141:105031. <http://dx.doi.org/10.1016/j.combiomed.2021.105031>.
- [10] Alswilem L, Asadov E. Deep learning in maize disease classification. *Artif Intell Appl Sci* 2025;1(1):20–7. <http://dx.doi.org/10.69882/adba.ai.2025074>.
- [11] Tan CS, Mohd-Mokhtar R, Arshad MR. A comprehensive review of coverage path planning in robotics using classical and heuristic algorithms. *IEEE Access* 2021;9:119310–36. <http://dx.doi.org/10.1109/ACCESS.2021.3108177>.
- [12] Zhao S, Hwang S-H. Complete coverage path planning scheme for autonomous navigation ROS-based robots. *ICT Express* 2024;10:83–9. <http://dx.doi.org/10.1016/j.icte.2023.06.009>.
- [13] Kim J. Autonomous robot vacuum system composed of a cleaner robot and a dust storage robot. *J Franklin Inst* 2024;361:106938. <http://dx.doi.org/10.1016/j.jfranklin.2024.106938>.
- [14] Manasa M, VTS, BV, Rao S, GPS. Smart vacuum cleaner. *Glob Transitions Proc* 2021;2:553–8. <http://dx.doi.org/10.1016/j.gltp.2021.08.051>.
- [15] Kumra S, Joshi S, Sahin F. GR-ConvNet v2: A real-time multi-grasp detection network for robotic grasping. *Sensors* 2022;22(16):6208. <http://dx.doi.org/10.3390/s22166208>.
- [16] Xiao X, Wang Y, Zhou B, Jiang Y. Flexible hand claw picking method for citrus-picking robot based on target fruit recognition. *Agric* 2024;14(1227):1–17. <http://dx.doi.org/10.3390/agriculture14081227>.
- [17] Yang T, Hwang J, Jeong J, Yoon M, Yoon S-E. Efficient navigation among movable obstacles using a mobile manipulator via hierarchical policy learning. 2025, arXiv preprint [arXiv:2506.15380](https://arxiv.org/abs/2506.15380).
- [18] Iyer SR, Pal A, Hu J, Adeleye A, Aggarwal A, Christensen HI. Household navigation and manipulation for everyday object rearrangement tasks. 2023, arXiv preprint [arXiv:2312.06129](https://arxiv.org/abs/2312.06129).
- [19] Gu J, Chaplot DS, Su H, Malik J. Multi-skill mobile manipulation for object rearrangement. In: *Proc. international conference on learning representations*. 2023.
- [20] An KY, Kwon H, Jang J-Y, Song K. Acoustic metamaterial design for noise reduction in vacuum cleaner. *J Mech Sci Technol* 2022;36(11):5353–62. <http://dx.doi.org/10.1007/s12206-022-1002-0>.
- [21] Panwar VS, Pandey A, Hasan ME. Design and fabrication of a novel concept-based autonomous controlled solar powered four-wheeled floor cleaning robot for wet and dry surfaces. *Int J Inf Technol* 2022;14(4):1995–2004. <http://dx.doi.org/10.1007/s41870-022-00893-1>.
- [22] Yapici NB, Tuglular T, Basoglu N. Assessment of human-robot interaction between householders and robotic vacuum cleaners. In: 2022 IEEE technology and engineering management conference. 2022, p. 204–9. <http://dx.doi.org/10.1109/TEMSCONEUROPE54743.2022.9802007>.
- [23] Grimme SA, Kollakidou A, Zarp CS, Hornecker E, Krüger N, Graf P, Marchetti E. Floor cleaners as helper pets: Projecting assistive robots' agency on zoomorphic affordances. *SN Comput Sci* 2023;4:372. <http://dx.doi.org/10.1007/s42979-023-01769-2>.
- [24] Eren A, Doğan H. Design and implementation of a cost-effective vacuum cleaner robot. *Turk J Eng* 2022;6(2):166–77. <http://dx.doi.org/10.31127/tuje.830282>.
- [25] Vishal R, Raghavan P, Rajesh R, Michael S, Elarab MR. Design of dual purpose cleaning robot. *Procedia Comput Sci* 2018;133:518–25. <http://dx.doi.org/10.1016/j.procs.2018.07.065>.
- [26] Li Z, Xu Q, Tam LM. A survey on techniques and applications of window-cleaning robots. *IEEE Access* 2021;9:111518–30. <http://dx.doi.org/10.1109/ACCESS.2021.3103757>.
- [27] Ambadkar P, Kumbhare P, Bonde C, Shukla H, Dharkar K, Vidhale B. AI-based smart cleaner with IoT integration. In: 2023 IEEE 8th international conference for convergence in technology. Pune, India; 2023, p. 1–6. <http://dx.doi.org/10.1109/I2CT57861.2023.10126417>.
- [28] Irawan Y, Muhandi M, Ordila R, Diandra. R. Automatic floor cleaning robot using arduino and ultrasonic sensor. *J Robot Control (JRC)* 2021;2(4):240–5. <http://dx.doi.org/10.18196/jrc.2485>.
- [29] Lin J, Kuo Y-M, Lin M, Lin C, Tsai P-Y, Lin C-W, Huang S-H, Chen C-C. Performance testing of cordless handheld vacuum cleaners. *Aerosol Air Qual Res* 2024;24(10). <http://dx.doi.org/10.4209/aaqr.240082>.
- [30] FakhrHosseini S, Lee C, Lee S-H, Coughlin J. A taxonomy of home automation: Expert perspectives on the future of smarter homes. *Inf Syst Front* 2024. <http://dx.doi.org/10.1007/s10796-024-10496-9>.